

The BEAM as a Digital Twin Substrate: A Technical Case for Elixir and OTP

Amin Jafarpour

School of Computing Science

Simon Fraser University

Burnaby, British Columbia, Canada

[author e-mail --- PLACEHOLDER]

ElixirWare Movement

Iron IoT Administration

(non-profit research organization)

Vancouver, British Columbia, Canada

[org contact --- PLACEHOLDER]

Abstract—Digital twin platforms are usually built by assembling a general-purpose runtime, a container orchestrator, an external state store, a message broker, and a supervisory control plane, and then spending most of the engineering budget reconciling the seams between them. This white paper argues that a substantial part of that assembly is redundant when the twin layer is built on the BEAM, the virtual machine underlying Erlang and Elixir, together with the Open Telecom Platform (OTP) libraries. The argument is structural rather than promotional. We first derive ten runtime requirements from the digital twin literature and from the ISO 23247 and ISO/IEC 30173 standards, covering entity multiplicity, state longevity, soft real-time responsiveness, partial failure containment, lifecycle churn, location transparency, observability, evolvability, edge symmetry, and security posture. We then examine the BEAM execution model — lightweight preemptively scheduled processes with private heaps, asynchronous message passing, supervision trees, transparent distribution, and hot code loading — and show that eight of the ten requirements correspond to primitives the runtime already provides, while the remaining two require deliberate engineering. We give a reference architecture that maps the four ISO 23247 domain entities onto OTP structures, present the design patterns and anti-patterns that determine whether such a system scales, and treat the limits honestly: numerical throughput, hard real-time guarantees, the full-mesh ceiling of Distributed Erlang, and the security consequences of a runtime that was designed for trusted networks. We include a correction to the widely repeated “nine nines” claim, whose evidentiary basis is weaker than its citation frequency suggests, and we outline an evaluation methodology grounded in the C-Town water distribution benchmark. The conclusion is conditional: the BEAM is an unusually strong substrate for the coordination, supervision, and state-custody layer of a digital twin system, and an unusually weak one for the numerical simulation layer, and a serious architecture should exploit the first while delegating the second.

Index Terms—digital twin, Elixir, Erlang, BEAM, OTP, actor model, ISO 23247, cyber-physical systems, edge computing, fault tolerance, industrial IoT

I. INTRODUCTION

A digital twin is a piece of software that is obliged to stay honest about something in the physical world. That obligation is what distinguishes it from a simulation, a dashboard, or a database of sensor readings. A simulation may run faster than real time and answer counterfactual questions; a dashboard displays whatever arrived most recently; a time-series store accumulates history. A twin must maintain a live correspondence with a physical counterpart, absorb the

counterpart’s failures and reconfigurations without losing that correspondence, and remain available for query and command throughout. Everything difficult about building twin platforms follows from that obligation.

The dominant industrial answer to this problem has been to compose a platform from independently designed parts. A stream processor ingests telemetry. A document or graph store holds twin state. A container orchestrator provides placement and restart. A broker provides routing. A separate control plane provides lifecycle management. Each part is individually mature, and the composite is individually defensible. The difficulty is that the properties the twin actually needs — that a given physical asset is represented by exactly one authoritative logical entity, that the failure of one entity does not disturb its neighbours, that state survives restart with defined semantics, that a command issued to a twin is either applied or observably rejected — are properties of the composition, not of any component. They must be established by integration work, re-established after every version change, and verified by tests that span process and network boundaries.

This paper argues that a different starting point removes a large fraction of that integration surface. The BEAM, the virtual machine that executes Erlang, Elixir, and Gleam, was designed for a problem with the same shape as the twin problem: maintaining large numbers of long-lived, independently failing, stateful entities that mirror physical devices, under continuous operation and without downtime for upgrade. That problem was telephone switching, and the solution — isolated lightweight processes, preemptive scheduling, supervision hierarchies, transparent distribution, and hot code replacement — was codified in the Open Telecom Platform libraries and validated over three decades of carrier deployment [1], [2]. The claim here is not that telephony and digital twins are the same domain. It is that they impose nearly the same requirements on a runtime, and that a runtime built to satisfy those requirements natively will require less scaffolding than one that must have them bolted on.

The argument is developed in a specific order and with deliberate restraint. Section II derives the runtime requirements from the twin literature and the governing standards, rather than from any implementation, so that the requirements can be applied to any candidate substrate. Section III describes

the BEAM execution model at the level of detail needed to evaluate the correspondence, including the mechanisms that practitioners most often misunderstand. Section IV performs the requirement-by-requirement comparison and is explicit about where the correspondence is exact, where it is partial, and where it fails. Sections V through VI give a concrete reference architecture, a mapping onto ISO 23247, and the design patterns that separate systems that scale from systems that merely start. Sections VII through X treat the uncomfortable material: throughput ceilings, clustering limits, the actual evidentiary status of the reliability folklore, and a security posture that has recently been shown to be less benign than the community assumed. Sections XI through XIII compare the approach with the credible alternatives and state plainly the conditions under which it is the wrong choice.

Two framing commitments shape the whole document. The first is that a technical case that omits the counter-case is advocacy, not analysis; every claim of suitability here is accompanied by the boundary at which it stops holding. The second is that architectural claims must be falsifiable. Section XIV therefore proposes an evaluation methodology, grounded in a public water distribution benchmark, that would allow the claims made here to be tested rather than merely asserted, and it is explicit that such an evaluation remains to be carried out at scale.

II. DIGITAL TWINS AND WHAT THEY DEMAND OF A RUNTIME

A. Definitional ground

The digital twin concept originates in Grieves’s product lifecycle management work and was subsequently formalized in a substantial engineering literature [9], [11], [12]. For present purposes the most operationally useful formulation is the taxonomy of Kritzing et al., which distinguishes three levels by the automation of data flow [10]. A digital model has manual data flow in both directions and is effectively a design artifact. A digital shadow has automatic flow from physical to digital and manual flow back, and describes the overwhelming majority of systems marketed as twins. A digital twin has automatic flow in both directions, which means the digital entity can act on its physical counterpart without a human in the loop.

This distinction is not pedantry; it determines the runtime requirements entirely. A shadow can tolerate eventual consistency, arbitrary restart semantics, and best-effort delivery, because a human reviews the output before anything physical happens. A true twin cannot, because a lost command, a duplicated command, or a command applied to a stale state estimate has physical consequences. The requirements derived below are the requirements of the third category. Systems that only need the second can be built on a much wider range of substrates, and an honest architecture should establish which category it is targeting before selecting technology.

The standards landscape sharpens this further. ISO 23247, published in four parts in 2021, provides a reference architecture for digital twins in manufacturing that decomposes a twin

system into four domain entities: the Observable Manufacturing Element, which is the physical asset and formally sits outside the framework because it already exists; the Data Collection and Device Control Entity, which acquires state from the element and issues control back to it; the Digital Twin Entity, which holds the digital representation and maintains synchronization; and the User Entity, which comprises the applications and people that consume and direct the twin [13], [17]. Part 4 additionally distinguishes the proximity network, connecting the device communication layer to the physical elements, from the access network, connecting that layer to the twin and user entities [14]. ISO/IEC 30173:2023 supplies vocabulary and concepts at a cross-domain level, and IEC 63278 specifies the Asset Administration Shell as an interoperable information carrier [15], [16]. The value of these documents for an architecture paper is that they fix the boundaries across which a runtime must provide guarantees, and they do so without prescribing an implementation technology.

B. Ten runtime requirements

From the definitional material and the reference architecture, ten requirements on the execution substrate follow. They are stated here in technology-neutral terms so that any candidate runtime can be assessed against them.

The first requirement is *entity multiplicity with isolation*. A twin system represents many physical elements — thousands in a building, hundreds of thousands in a distribution network, potentially millions in a fleet — and each must be an addressable logical entity with its own state and its own failure domain. Multiplicity alone is easy; multiplicity with isolation is the hard part, because the natural implementations either share a heap and therefore share failures, or allocate an operating system process or container per entity and therefore become uneconomic at three or more orders of magnitude.

The second is *state longevity with defined restart semantics*. A twin’s state is not a cache. It accumulates over the asset’s operational life and often cannot be reconstructed from history alone, because the history is lossy, the reconstruction is expensive, or the twin’s internal estimate depends on a filter with its own state. The runtime must therefore support entities that live for months, and the platform must define precisely what happens to that state on crash, on planned restart, on upgrade, and on migration.

The third is *soft real-time responsiveness under load*. Twin systems are almost never hard real-time — that responsibility remains with the programmable logic controller or the embedded firmware — but they are strongly latency-sensitive in the tail. A twin that answers in five milliseconds at the median and eight seconds at the ninety-ninth percentile is operationally useless for anomaly detection, because the tail is precisely where the anomalies concentrate [27]. What is required is not minimal latency but bounded latency dispersion under sustained and uneven load.

The fourth is *partial failure as a normal condition*. Sensors fail, gateways lose power, network segments partition, and individual twins encounter malformed inputs that crash their

update logic. In a system with a hundred thousand entities, some non-empty subset is in a failed state at all times. The runtime must make partial failure a routine event with local consequences rather than an exceptional one with global consequences.

The fifth is *lifecycle churn*. Physical assets are commissioned, decommissioned, replaced, and reconfigured continuously. Twins are therefore created and destroyed at runtime, not enumerated at deployment time. A runtime in which creating an entity costs tens of milliseconds and a configuration reload imposes an architecture that cannot follow the physical plant.

The sixth is *location transparency and mobility*. As the twin population grows beyond one machine, entities must be reachable by logical identity rather than by network address, and must be relocatable between machines without their correspondents learning of the move. Without this property, sharding decisions leak into application code and become effectively permanent.

The seventh is *introspection of live state*. Operators must be able to ask what a twin currently believes, why it believes it, and what it has recently done, on a running production system, without a redeploy and without a debugger that stops the world. In a system whose entire purpose is correspondence with physical reality, the inability to inspect the digital side in situ is a defect of the first order.

The eighth is *evolvability without loss of correspondence*. Twin logic changes far more often than the physical assets do: new models, new inference rules, new derived quantities. If updating that logic requires draining and restarting the twin population, then correspondence is broken on every release, and the release cadence becomes a function of how much correspondence loss the operator can tolerate.

The ninth is *edge and cloud symmetry*. Latency, bandwidth cost, and regulatory constraint all push part of the twin population toward the physical plant, while aggregate analysis pulls it toward the centre. A substrate that forces two different programming models — one for the gateway, one for the cluster — imposes a permanent translation cost at exactly the boundary where correctness matters most.

The tenth is *a defensible security posture*. A true digital twin is an actuation path into physical infrastructure. Compromise of the twin layer is compromise of the plant. The runtime's default trust assumptions, its deserialization behaviour, its code loading facilities, and its inter-node authentication are therefore security-critical in a way that they are not for a stateless web service.

These ten requirements are used throughout the remainder of the paper as the evaluation frame. They are numbered R1 through R10 in the order presented.

III. THE BEAM EXECUTION MODEL

The BEAM is a register-based virtual machine executing bytecode compiled from Erlang, Elixir, Gleam, and several smaller languages, with an ahead-of-time native code path (BeamAsm) introduced in OTP 24 that substantially narrowed the gap to compiled languages for sequential code. As of

the current release line, Erlang/OTP 29 and Elixir 1.20 are the reference versions, the latter having extended its set-theoretic type system to all language constructs with inference for function definitions [45]. The runtime is a realization of the actor model in the sense of Hewitt and Agha — independent entities communicating solely by asynchronous message passing [7], [8] — with the significant addition that isolation and fairness are enforced by the virtual machine rather than by convention. Four properties of the runtime matter for the argument that follows, and the authoritative account of each is the runtime system reference manual [38].

A. Processes

The BEAM's unit of concurrency is the process, which is not an operating system process or thread but a runtime-managed entity with its own private heap, its own stack, its own mailbox, and its own garbage collector. Creation costs a few hundred machine words — on the order of two to three kilobytes on a 64-bit system for a process that has not yet allocated — and a few microseconds. The default ceiling on simultaneously existing processes is roughly one million and is configurable into the hundreds of millions, subject to available memory.

Two consequences follow, and they are the load-bearing consequences for this paper. The first is that per-entity concurrency becomes economically ordinary rather than exotic. A system that models one hundred thousand sensors with one hundred thousand processes is unremarkable on the BEAM and is architecturally prohibitive under a thread-per-entity or container-per-entity model. The second is that because each process has a private heap, a process crash destroys only that process's memory. There is no shared mutable state to leave inconsistent, and therefore no need for the defensive coding that dominates systems where one entity's corruption is every entity's problem.

Garbage collection is likewise per-process and generational. Because heaps are small and independent, collection pauses are short and local; the runtime never stops all processes to collect. This is the mechanism underlying the BEAM's latency characteristics, and it is a different mechanism from the low-pause-collector strategies used by managed runtimes with a shared heap, with different failure modes and different tuning surfaces.

B. Preemptive scheduling

The BEAM runs one scheduler thread per logical core by default, each with its own run queue and with work stealing between queues. Scheduling is preemptive and is driven by a reduction counter: each process is allotted a budget of roughly four thousand reductions — a unit approximating function calls and other work — after which it is descheduled regardless of what it is doing. Blocking operations, including message receives and many I/O operations, yield control.

This is the property most often underappreciated by engineers arriving from other actor systems. Cooperative and thread-pool-based actor runtimes are vulnerable to a single actor monopolizing a carrier thread through a tight loop or a blocking

call, and the standard mitigations — separate dispatchers, bounded blocking pools, discipline about what may be called from an actor — are conventions that must be enforced by review. On the BEAM the guarantee is enforced by the virtual machine. One misbehaving twin cannot starve the other ninety-nine thousand. The exception, which matters and is treated in Section XIII, is native code invoked through a Native Implemented Function, which executes outside the reduction accounting and can indeed block a scheduler.

C. Message passing and shared-nothing semantics

Processes communicate only by asynchronous message passing. A message sent between processes on the same node is copied into the recipient’s heap, with the notable exception of binaries larger than sixty-four bytes, which are reference-counted and held off-heap. Copying is a real cost and is the reason that a naive design passing large payloads between many processes performs poorly; it is also the reason that no process can observe another process’s partially updated state.

There is a deliberate escape hatch. Erlang Term Storage provides in-memory tables with concurrent read and write access outside any single process heap, and `persistent_term` provides a read-optimized store for data that changes rarely. These are the correct tools for twin registries, configuration, and lookup indices, and the incorrect tool for twin state itself, for reasons developed in Section VI.

D. Supervision, distribution, and code loading

OTP supplies a set of behaviours — `gen_server`, `gen_statem`, `gen_event`, `supervisor` — that encode the generic parts of process structure and leave only the domain-specific callbacks to the application [3]–[6]. Supervisors monitor children, restart them according to declared strategies, and are themselves supervised, producing a tree in which the failure of a leaf is handled by its parent and the failure of a subtree is handled further up. This is the architectural expression of the crash-only philosophy articulated independently by Candea and Fox: recovery paths that are exercised on every failure are more trustworthy than recovery paths reserved for exceptional conditions [25]. Coupled with Gray’s older observation that most production failures are transient and non-deterministic [26], restart-to-a-known-state is not a crude fallback but the statistically correct default.

Distribution is built into the runtime. Nodes connect into a cluster, and a process on one node sends to a process on another with the same syntax and semantics used locally, subject only to the additional failure mode of the connection dropping. Process monitors and links propagate across nodes, so remote failure is observable through the same mechanism as local failure. The default topology is a fully connected mesh, which is the origin of the scalability ceiling discussed in Section VIII.

Finally, the runtime supports loading a new version of a module while the old version continues to execute, with two versions of a module resident simultaneously and processes migrating to the new version at defined points. Managed carefully, this permits logic to be replaced without terminating

the processes that use it. Managed carelessly, it is an operational hazard and a security concern, and both aspects are addressed below.

IV. THE CORRESPONDENCE ARGUMENT

The case for the BEAM as a twin substrate rests on the claim that the runtime already provides, as primitives, the properties that other substrates must assemble. This section examines that claim requirement by requirement and marks each as satisfied by construction, satisfied with engineering, or not satisfied.

R1, entity multiplicity with isolation, is satisfied by construction and is the strongest single point of the argument. The natural BEAM design assigns one process per twin. Isolation is not a discipline imposed on the design; it is a property of the process abstraction. The economics work at the scale twin systems require, and the failure domain boundary coincides exactly with the entity boundary that the domain already defines. No other mainstream runtime gives all three of these simultaneously.

R2, state longevity with defined restart semantics, is satisfied with engineering. A BEAM process holds state in its own heap indefinitely at negligible cost, and a twin that has run for six months is not unusual. What the runtime does not provide is durability: process state is lost on crash and on node failure. The platform must therefore choose a state-recovery discipline, and the choice is a first-class architectural decision rather than a detail. The candidates are event sourcing with replay, periodic snapshotting to durable storage with bounded replay from a journal, and write-through to an external store on each accepted update. Each has different recovery-time and consistency properties, and Section VI takes a position.

R3, soft real-time responsiveness under load, is satisfied by construction, and this is the property that most distinguishes the BEAM from superficially similar alternatives. Preemptive reduction-based scheduling plus per-process garbage collection produce latency distributions with short tails under heterogeneous load. The guarantee is not that any individual operation is fast — sequential BEAM code is slower than equivalent C, Rust, or well-tuned JVM code — but that the slowness of one operation does not propagate into the latency of unrelated operations. For a twin platform, where the query that matters is the one issued during the incident, tail behaviour dominates median throughput in operational value.

R4, partial failure as a normal condition, is satisfied by construction and is the requirement OTP was built to address. A twin whose update logic encounters a malformed reading crashes, its supervisor restarts it into a known state, the failure is logged with the crashing state attached, and the other twins never observe the event. The restart intensity limits in the supervisor specification convert repeated local failure into escalated failure at a higher level, which is the correct behaviour: a twin that crashes twenty times in ten seconds has a systematic problem, not a transient one, and should not be restarted in a tight loop.

R5, lifecycle churn, is satisfied by construction. `DynamicSupervisor` exists precisely to start and stop

supervised children at runtime, and the cost of commissioning a twin is the cost of spawning a process and registering a name — microseconds, not the seconds required to schedule a container or the minutes required for a configuration-driven redeploy. A twin platform on the BEAM can track the physical plant’s commissioning cadence exactly.

R6, location transparency and mobility, is satisfied with engineering, and the engineering is non-trivial. Distributed Erlang gives transparent addressing of a known process identifier on a known node, and `global` gives cluster-wide name registration, but neither gives automatic placement, rebalancing, or handoff. Libraries fill the gap: `Horde` provides a distributed supervisor and registry built on `delta-CRDTs` [21], `Swarm` provides clustering with process handoff, and `Erleans` imports the virtual actor model from Orleans, in which entities are addressed by logical identity and activated on demand by the runtime [19], [20]. The virtual actor pattern is a particularly good fit for twins, because a twin has a natural durable identity independent of whether it currently has a live process, and because a twin population is typically far larger than the working set active at any moment. The qualification is that these libraries have real semantics — eventual consistency of the registry, a window during which two activations can exist after a partition — and a design that assumes single activation without enforcing it will produce duplicate commands to physical equipment. This is a correctness matter, not a performance matter.

R7, introspection of live state, is satisfied by construction and is underrated. A running BEAM node can be attached to from a remote shell, and the state of any process inspected with `sys:get_state/1`, its message queue length and memory examined, its execution traced with the built-in trace mechanism, and its behaviour observed through `observer` or the lower-overhead `recon` toolkit. The `telemetry` library and the OpenTelemetry Erlang and Elixir implementations provide the structured export path to conventional monitoring. For a twin platform this is not a convenience: the ability to ask a specific twin what it believes, on the production system, during an incident, without redeploying, directly addresses the central operational question of whether the digital and physical sides still agree.

R8, evolvability without loss of correspondence, is satisfied with engineering and carries meaningful operational cost. Hot code loading permits replacing twin logic while twin processes and their accumulated state remain live, which is exactly the requirement. Doing so safely requires implementing state migration callbacks, constructing release upgrade artifacts, and testing upgrade paths — a discipline that many teams reasonably decline in favour of rolling restarts with state recovery. The honest framing is that the BEAM makes zero-downtime logic replacement possible where most runtimes make it impossible, and that exercising the capability is a deliberate investment rather than a free benefit.

R9, edge and cloud symmetry, is satisfied by construction to an unusual degree and is treated in detail in Section XI. The same OTP abstractions run on a cloud node, on a Linux-

class gateway through Nerves, and on a microcontroller through AtomVM or GRiSP. A twin’s supervision structure and message protocol are expressed once.

R10, security posture, is *not* satisfied by construction, and this paper takes the position that presenting it otherwise would be irresponsible. Distributed Erlang’s default authentication is a shared secret cookie transmitted over an unencrypted connection, and a node that joins a cluster can execute arbitrary code on every other node. The runtime’s deserialization primitive will construct atoms from untrusted input unless explicitly restricted, giving a denial-of-service path through atom table exhaustion. Hot code loading is a code injection primitive by design. And CVE-2025-32433, a pre-authentication remote code execution flaw in the OTP SSH server rated CVSS 10.0 and observed being exploited in the wild, demonstrated that the ecosystem’s security assumptions require the same scrutiny applied to any other platform [30], [31]. Section X treats each of these and the corresponding mitigations.

The aggregate result is six requirements satisfied by construction, three satisfied with deliberate engineering, and one requiring active defensive work that the runtime does not perform on the architect’s behalf. That is a strong result, and it is not the same as an unqualified one.

V. A REFERENCE ARCHITECTURE IN ELIXIR AND OTP

The requirements analysis suggests a shape. This section makes it concrete, first as a layered architecture and then as an explicit mapping onto the ISO 23247 reference entities.

A. Layers

At the boundary with the physical plant sits the *acquisition layer*. Its responsibility is to accept telemetry from heterogeneous sources — MQTT brokers, OPC UA servers, Modbus polling loops, HTTP webhooks, serial links — normalize it into a canonical internal representation, and deliver it to the correct twin. The critical property is back-pressure. Telemetry arrival rates are set by the physical world and are not negotiable; if the twin layer cannot keep up, the system must shed load or buffer deliberately rather than accumulate unbounded message queues. `GenStage` and `Broadway` implement demand-driven flow control natively, with connectors for the common brokers [43], and this is the correct tool. A design in which a protocol handler sends directly to twin processes without a demand signal has no back-pressure mechanism at all, and will fail under exactly the burst conditions — a plant upset, a reconnection storm — during which the twin layer is most needed.

Above it sits the *twin layer*, which is the architectural centre. Each observable physical element is represented by one supervised process holding that element’s current state estimate, its recent history window, its configuration, and its command interface. The process is implemented as a `gen_statem` rather than a `gen_server` whenever the element has meaningful operational modes, because twins almost always do: a pump is stopped, starting, running, or faulted, and the legal transitions and the legal commands differ by mode. Encoding those modes in the state machine makes illegal transitions a

runtime error caught by the supervisor rather than a silent corruption of the state estimate. Twins are registered in a `Registry` keyed by the element’s durable identifier, so that all addressing is by physical identity rather than by process identifier, and they are started by a `DynamicSupervisor` so that commissioning and decommissioning are ordinary runtime operations. For populations large enough that a single registry or supervisor becomes a contention point, `PartitionSupervisor` shards both across schedulers.

Beside the twin layer sits the *persistence layer*, whose responsibility is to make twin state survive the loss of the process holding it. The position taken here is that twin state should be persisted as an event journal with periodic snapshots, rather than as mutable rows updated in place. The reason is domain-specific rather than fashionable: a twin’s obligation is correspondence, and when correspondence is later found to have been lost, the operational question is invariably *when* and *on what evidence* the twin came to believe something false. An event journal answers that question directly and a mutable current-state row does not. PostgreSQL with the TimescaleDB extension serves both the journal and the telemetry history well, accessed through `Ecto`; the snapshot cadence is tuned to bound recovery time.

Above the twin layer sits the *serving layer*, which answers queries, accepts commands, and publishes change notifications. Phoenix provides the HTTP and WebSocket surface, `Phoenix.PubSub` the internal publish-subscribe fabric, and `Phoenix LiveView` a server-rendered interactive interface in which a twin’s state changes propagate to a browser without a separate front-end application and without a polling loop [44]. For twin systems specifically, `LiveView` is a larger practical advantage than it first appears, because the natural interface to a twin population is a live view of current state, and building that conventionally requires a second application, a second state model, and a synchronization protocol between them.

The *analysis layer* sits outside the twin processes and is where this architecture deliberately does not use the BEAM for the heavy work. Physics simulation, model fitting, and machine learning inference are dispatched to `Nx` with the `EXLA` backend, which compiles numerical computations through `XLA` to CPU or GPU [42], or to external services entirely. Section XIII argues that this delegation is not a workaround but the correct architecture.

B. Mapping onto ISO 23247

The correspondence with the ISO 23247 reference architecture is close enough to be useful as a design discipline. The Observable Manufacturing Element remains the physical asset and has no software representation, exactly as the standard intends. The Data Collection and Device Control Entity maps onto the acquisition layer together with the command-emission path: in OTP terms, a supervised subtree of protocol adapters and `Broadway` pipelines per protocol, with the proximity network terminating at those adapters. The Digital Twin Entity maps onto the twin layer and its persistence: the standard’s sub-entities for representation, synchronization, and simulation

correspond respectively to the process state, the update and reconciliation logic, and the dispatched numerical work. The User Entity maps onto the serving layer. The access network is the boundary between the acquisition supervision subtree and the twin supervision subtree, and in a distributed deployment it may be an actual network boundary between BEAM nodes.

Two observations follow from performing this mapping explicitly. The first is that the standard’s entity decomposition and OTP’s supervision decomposition are both partitions by failure domain, which is why they align so readily; a supervision tree whose top-level children are the four entities is a defensible starting structure. The second is that the standard’s insistence on a distinguished device control path — rather than treating control as the inverse of collection — maps onto a design decision with security consequences, since the control path is the actuation path and should be supervised, authenticated, and audited separately. This is developed in Section X. Appendix A summarizes the mapping.

VI. DESIGN PATTERNS AND ANTI-PATTERNS

The difference between a BEAM twin platform that scales and one that does not is rarely the runtime. It is a small number of design decisions, each of which has a correct and an incorrect resolution that are not obvious in advance.

A. The twin process and its reconciliation loop

The core pattern is a state machine per element with an explicit reconciliation step. The twin holds a reported state, derived from telemetry; a desired state, derived from commands; and the reconciliation logic that decides what to actuate. This is the device shadow model familiar from major IoT platforms, and its virtue is that it makes the twin’s obligation explicit: the delta between desired and reported is the twin’s outstanding work, and a twin whose delta is persistently non-zero is a twin whose correspondence has been lost.

```
defmodule Plant.Twin do
  @behaviour :gen_statem
  # states: :unsynced | :nominal | :degraded | :faulted

  def start_link(id), do:
    :gen_statem.start_link(via(id), __MODULE__, id, [])

  def callback_mode, do: [:state_functions, :state_enter]

  def init(id) do
    {:ok, :unsynced, State.load(id), [[:state_timeout,
      @sync_deadline_ms, :never_synced]]}
  end

  # Telemetry updates the reported half of the shadow.
  def nominal(:cast, {observed, reading}, s) do
    s = State.apply_observation(s, reading)
    :ok = Journal.append(s.id, {observed, reading})
    case Reconcile.delta(s) do
      :none -> {:keep_state, s}
      delta -> {:keep_state, s, [[:next_event, :internal,
        {:actuate, delta}]]}
    end
  end

  # No reading within the deadline is itself information.
  def nominal(:state_timeout, :stale, s), do:
    {:next_state, :degraded, State.mark_stale(s)}
end
```

Three details in that sketch carry the weight. The journal append happens before actuation, so that a crash between deciding and acting leaves evidence. The absence of a reading is handled by a state timeout and treated as a state transition, because silence from a physical asset is information and a twin that silently retains a stale estimate has failed at its only job. And the actuation decision is expressed as an internal event rather than a direct call, keeping the state machine the single point at which the twin’s behaviour is determined.

B. Registry, supervision, and identity

Twins must be addressed by the durable identity of the physical element, never by process identifier, because process identifiers change on every restart and a design that caches them has embedded a fault into every recovery path. The standard structure is a `Registry` for local addressing and a `DynamicSupervisor` for lifecycle, wrapped so that a caller can address a twin that is not currently running and have it activated on demand.

```
defmodule Plant.Twins do
  def via(id), do: {:via, Registry, {Plant.Registry, {:twin, id}}}

  # Address by physical identity; activate lazily.
  def whereis_or_start(id) do
    case Registry.lookup(Plant.Registry, {:twin, id}) do
      [{pid, _}] -> {:ok, pid}
      [] -> DynamicSupervisor.start_child(
        Plant.TwinSup, {Plant.Twin, id})
    end
  end
end
```

Lazy activation matters at scale for a reason specific to twin workloads: the population of commissioned assets is typically far larger than the population generating telemetry at any moment. A water network has tens of thousands of modelled junctions and a few hundred actively instrumented ones; a building has thousands of modelled spaces and a working set determined by occupancy. Keeping a process resident for every modelled element is possible on the BEAM but is rarely the best use of memory, and the virtual actor pattern — activate on first reference, passivate after a period of inactivity, rehydrate from the journal — gives a much better ratio. This is the pattern `Erleans` implements directly, and the pattern that Orleans established [18], [19].

C. Anti-patterns

Four failure modes recur often enough to name.

The first is *ETS as the system of record*. Erlang Term Storage is fast, shared, and tempting, and using it to hold twin state discards precisely the isolation property that motivated the architecture. State in ETS is mutable by any process that has the table reference; a corrupting write has no owner and no supervisor; and a crash leaves the table populated with the corrupt value rather than reverting to a known state. ETS is the right tool for registries, indices, caches, and reference data, and the wrong tool for the authoritative state of an entity that has a physical counterpart.

The second is *the twin as synchronous bottleneck*. A twin process executes one message at a time. If a twin’s handler performs a synchronous call to an external service — a database write, an HTTP request to a historian, a model inference — then that twin’s throughput is bounded by that service’s latency, and the twin’s mailbox grows without limit under sustained telemetry. The correct structure keeps the twin’s handlers fast and local, and delegates slow work to a `Task` under a supervisor, with the result returned as a message. The twin remains responsive to queries and to further telemetry while the slow work proceeds.

The third is *unbounded mailboxes*. The BEAM does not bound process mailboxes; a process receiving faster than it can process will consume memory until the node dies, and the failure presents as gradual degradation followed by sudden collapse. Twin systems are unusually exposed to this because arrival rates are set by the physical world. The mitigation is demand-driven ingestion throughout, plus explicit monitoring of mailbox lengths as a first-class operational metric with alerting well before saturation.

The fourth is *assuming single activation without enforcing it*. In a clustered deployment with a CRDT-based registry, a network partition can produce two live activations of the same twin, each accepting commands. For a shadow this is a data quality problem; for a true twin it is two commands to one physical actuator. The mitigation is to make the actuation path, not the twin process, the point of serialization — for instance by routing all commands for a given element through a consistently-hashed owner with a fencing token, so that a stale activation’s commands are rejected at the boundary. The architectural principle is that the safety property must be enforced where the physical consequence occurs, not where the logical entity happens to live.

VII. SCALE, PERFORMANCE, AND HONEST LIMITS

The BEAM’s performance profile is unusual and is frequently misdescribed in both directions. Stating it precisely is necessary for the argument to be credible.

Sequential numerical throughput is poor relative to C, Rust, Go, or a well-warmed JVM. The `BeamAsm` just-in-time compiler introduced in OTP 24 materially improved this, but the gap on tight numeric loops remains large, typically an order of magnitude or more. Any architecture that places finite-element analysis, computational fluid dynamics, large matrix operations, or neural network inference inside BEAM processes will be disappointed, and the disappointment is a property of the runtime rather than of the implementation.

Concurrency throughput and latency dispersion, by contrast, are excellent. The system sustains very large numbers of simultaneously live processes, schedules them fairly under preemption, and produces latency distributions whose tails are short even under uneven load. For the twin layer’s actual work — receiving a reading, updating a state estimate, evaluating a rule, deciding whether to act — this is the profile that matters, and the arithmetic involved is trivial.

The architectural conclusion is that the two profiles should be exploited separately. The BEAM holds identity, state, lifecycle, coordination, and supervision; the numerical work is dispatched elsewhere. Three delegation paths exist and differ in their risk. `Nx` with the `EXLA` backend compiles numerical expressions through `XLA` and runs them on CPU or GPU, keeping the orchestration in Elixir; this is the lowest-friction option and is appropriate for tensor-shaped work. Rust NIFs written with `Rustler` give near-native performance for custom algorithms, with the crucial caveat that a NIF executes outside reduction accounting: a long-running NIF blocks its scheduler and defeats the fairness guarantee that motivated the runtime choice in the first place. NIFs must therefore either complete in well under a millisecond, or yield explicitly, or run on a dirty scheduler. A NIF that crashes takes the entire node with it, so this path also punctures the isolation property. The third path — ports and external services — is slower but preserves both fairness and isolation, because the external program is a separate operating system process whose failure is observable rather than fatal.

Memory behaviour deserves a similar caution. Message passing copies, so a twin that receives a large payload on every update pays a copy on every update; the mitigation is to pass large data as reference-counted binaries or to keep it in ETS and pass a key. Reference-counted binaries can also produce a characteristic memory leak pattern, in which a small sub-binary retains a large parent binary, and the diagnosis of this pattern is a standard part of operating BEAM systems at scale.

VIII. DISTRIBUTION, CLUSTERING, AND CONSISTENCY

Distributed Erlang gives location-transparent messaging with failure detection across nodes, which is a substantial primitive to have for free. Its limits are equally concrete and must be designed around rather than discovered.

The default topology is a fully connected mesh: every node maintains a connection to every other node, with heartbeat traffic on each. Connection count grows quadratically, and practical experience places the comfortable ceiling in the low hundreds of nodes, well below what a large twin deployment might eventually want. The `RELEASE` project examined this ceiling in detail and proposed scalable distribution mechanisms [24], and `Partisan` addresses it directly by making the network overlay a runtime choice rather than a fixed mesh; the authors report roughly an order of magnitude increase in achievable node count together with substantial throughput and latency improvements over Distributed Erlang [22]. For a twin platform expecting to exceed a hundred nodes, adopting `Partisan` or partitioning into federated clusters is a design decision to make early, because retrofitting it means revisiting every assumption about global addressing.

Node membership must be managed. `libcluster` provides discovery strategies for the common deployment environments. Cluster-wide process registration is available through `global`, which is strongly consistent and serializes registration through a global lock, or through `Horde`, which is eventually consistent and built on delta-CRDTs [21]. The trade-off is the

familiar one and it has a domain-specific resolution: for twin registries, the eventual-consistency option is usually correct for throughput reasons, provided the safety property of single actuation is enforced at the actuation boundary as argued in Section VI rather than being assumed from the registry.

Partition behaviour requires an explicit position. When a cluster partitions, each side may believe it owns a given twin. Distributed Erlang’s default behaviour on partition healing is to keep both sides running in an inconsistent state; the runtime detects the partition but does not resolve it. A twin platform must choose, per twin class, whether availability or consistency is preferred during a partition, and must implement that choice [28]. The choice is genuinely domain-dependent: a temperature twin that continues serving a possibly-stale reading on the minority side is usually harmless, and a valve twin that continues accepting actuation commands on the minority side is not. A platform that applies one policy uniformly has made an error for at least one of these cases.

IX. FAULT TOLERANCE WITHOUT FOLKLORE

The reliability argument for the BEAM is strong and is habitually made badly. Correcting the standard overstatement makes the real argument more persuasive, not less.

The claim in circulation is that Ericsson’s AXD301 switch, written largely in Erlang, achieved “nine nines” of availability, that is 99.999999 percent, or roughly thirty-one milliseconds of downtime per year. The figure appears in conference presentations, blog posts, and a great deal of promotional material. Its evidentiary basis is thin, and Armstrong’s own thesis says so: the information came from a slide deck reporting that a major customer had run an eleven-node system at that availability, with no documented account of how the figure was obtained, and Armstrong noted explicitly that evidence for long-term operational stability had not been collected systematically [1]. Practitioners inside the community made the same objection at the time, observing that the AXD301’s availability also reflected redundant hardware, a substantial body of C code, and considerable conventional reliability engineering, none of which is attributable to the language [29]. A twin platform architect who cites nine nines in a design review should expect, and deserve, to be challenged.

The defensible claim is different in kind and stronger in practice. It is that the BEAM and OTP provide a structural approach to failure that has been validated across three decades of production systems in telecommunications, messaging, and infrastructure software: failures are contained within process boundaries by construction; recovery is performed by supervisors along declared strategies rather than by ad hoc exception handling; the recovery path is the same path exercised on every ordinary failure, so it is continuously tested rather than reserved for disaster [25]; and the empirical character of production faults — predominantly transient, timing-dependent, and non-deterministic — makes restart-to-known-state the statistically appropriate default response [26]. None of this delivers a particular number of nines. All of it lowers the engineering cost of reaching whatever availability target the

deployment actually requires, and it does so by making the failure-handling structure explicit and reviewable rather than diffuse.

For a twin platform, the practical consequence is specific. Availability of the platform as a whole is less interesting than availability of correspondence per twin, which is the property that determines whether an operator can trust what the system reports. That property is best served by an architecture in which a twin’s failure is local, its recovery is automatic and bounded, its state after recovery is defined, and the interval during which correspondence was lost is recorded and queryable. Each of those four is directly supported by the OTP structure described above. That is the reliability claim worth making.

X. SECURITY CONSIDERATIONS

A true digital twin holds an actuation path into physical infrastructure, which places the twin layer inside the operational technology trust boundary and makes its security posture a safety matter. The BEAM’s defaults were designed for trusted, physically secured telecommunications networks, and they have not been silently modernized. Several require deliberate attention.

Distribution authentication. Distributed Erlang authenticates peers with a shared secret cookie, transmitted in the clear on an unencrypted connection by default. Any party who learns the cookie and can reach the distribution port becomes a cluster member, and a cluster member can execute arbitrary code on every other node — this is not an exploit but the intended semantics of distribution. The mitigations are non-optional for any deployment reachable from an untrusted network: enable TLS distribution with mutual certificate authentication rather than relying on the cookie, bind the distribution and EPMD ports to management interfaces only, treat the cookie as a high-value secret with rotation, and segment the twin cluster from both enterprise IT and the plant network. In an operational technology context this segmentation argument should be made in the vocabulary of the applicable zone-and-conduit model, because that is the language the plant’s own security review will use.

Deserialization. The runtime’s binary term decoding will create atoms from input unless the safe option is specified. Atoms are not garbage collected and the atom table has a fixed maximum, so an attacker who can supply arbitrary terms can exhaust it and halt the node. The same category of error occurs in application code that converts external strings to atoms directly rather than converting only to already-existing atoms. Both are straightforward to eliminate and both appear regularly in code review.

Code loading. Hot code loading is a code injection primitive with a supported API. Its legitimate use in a twin platform is real, but the loading path must be authenticated, the artifacts signed, the operation audited, and the capability disabled entirely on edge deployments in physically accessible locations, where an attacker with local access and a distribution connection otherwise has a direct route to arbitrary code execution on the plant network.

Ecosystem vulnerabilities. The disclosure of CVE-2025-32433 in April 2025 — a pre-authentication remote code execution flaw in the OTP SSH server, rated CVSS 10.0, arising from the daemon processing connection protocol messages before authentication completed — is instructive beyond its own severity [30]. Exploitation attempts against internet-reachable systems were observed within weeks, and the affected component is embedded in widely deployed infrastructure software and in industrial equipment where patching cadence is slow [31]. The lesson for a twin architect is that the BEAM ecosystem is subject to the same vulnerability lifecycle as any other, that a runtime’s reputation for reliability confers no protection against memory-safety-adjacent or state-machine flaws in its libraries, and that software bills of materials and patch tracking apply here exactly as elsewhere.

Tooling. Static analysis for BEAM languages has a research base worth using. Dialyzer performs success-typing analysis across a whole application and finds a meaningful class of defect; Elixir 1.20’s set-theoretic type system now covers all language constructs and infers types for function definitions, catching a further class at compile time. The RefactorErl work from the Eötvös Loránd University group provides deeper static analysis and program comprehension for Erlang, and formal verification efforts on Core Erlang semantics supply a foundation for reasoning about program equivalence and refactoring correctness [32], [33]. For a twin platform whose failures have physical consequences, the marginal cost of running these tools in continuous integration is small against the marginal benefit.

Finally, there is a constructive relationship between the two subjects worth noting. Digital twins are increasingly used as a security instrument in their own right — as a high-fidelity model against which to detect anomalous process behaviour, and as a safe environment in which to evaluate attack scenarios against operational technology without touching production. A twin platform whose own security posture is weak cannot credibly serve that role, which is an additional reason for the twin layer’s controls to be at least as strong as those of the systems it observes.

XI. ONE PROGRAMMING MODEL FROM CLOUD TO EDGE

Requirement R9 asked for symmetry between edge and cloud, and this is the area where the BEAM’s position is most unusual. Three deployment targets share the same process model, the same supervision semantics, and the same message protocols.

Nerves builds a minimal Linux system around the BEAM for single-board computers and industrial gateways, producing a firmware image with no general-purpose operating system distribution beneath it [39]. Boot times are on the order of a few seconds, the root filesystem is read-only, and updates are delivered as complete signed firmware images with A/B partition switching and automatic rollback — a model far better suited to unattended field equipment than package-manager-based updates. For a twin platform, a *Nerves* gateway can run the same twin processes as the central cluster, holding local

twins for locally attached equipment and forwarding aggregates upward, with the partitioning between local and central twins being a deployment decision rather than a rewrite.

GRiSP targets bare-metal embedded boards, running the BEAM without an operating system underneath and providing direct access to hardware interfaces from Erlang, with real-time characteristics unavailable through a general Linux kernel [41]. It has been used as the substrate for research on resilient edge computation, including work on coordination-free programming models for partitioned networks [23].

AtomVM is a reimplementation of the BEAM for microcontrollers, running on ESP32, STM32, and Raspberry Pi Pico class hardware with tens or hundreds of kilobytes of memory [40]. It supports a substantial subset of the runtime, including processes, message passing, and a reduced OTP, which means that a sensor node can host actual supervised processes rather than a hand-written event loop. The architectural implication is significant: the leaf of the twin hierarchy can itself be an actor addressable by the same mechanisms as its parents, and the boundary between “device” and “twin” becomes a placement decision rather than a paradigm change. Peripheral access on such a node can be modelled as supervised, location-transparent processes reachable from remote nodes over Erlang distribution rather than over an application-level protocol, which collapses a translation layer that conventional architectures must build and maintain.

The value of this symmetry for twin systems is concrete rather than aesthetic. The decision about which twins run at the edge and which run centrally is driven by latency budgets, bandwidth costs, connectivity reliability, and data residency rules, all of which change over a system’s life. When edge and centre share a programming model, that decision remains revisable. When they do not, the initial partitioning tends to become permanent, because moving a twin across the boundary means reimplementing it.

XII. ALTERNATIVES CONSIDERED

An architecture argument is only as good as its treatment of the alternatives. Five are credible.

Akka and Apache Pekko on the JVM. The actor model on the JVM, with cluster sharding, persistence, and distributed data, is the closest technical analogue and in several respects more mature: cluster sharding with automatic rebalancing is a first-class supported feature rather than a community library, event-sourced persistence is well developed, and the surrounding JVM ecosystem for numerical and enterprise integration is vastly larger. The differences that matter are three. Scheduling is cooperative over a thread pool, so a blocking call in an actor can starve others and the mitigation is discipline rather than runtime enforcement. Garbage collection is over a shared heap, so pause behaviour is global rather than per-entity, which is the property that most directly affects tail latency in a twin workload. And in September 2022 Lightbend moved Akka from Apache 2.0 to the Business Source License for releases from 2.7 onward, requiring a commercial license for production use above a revenue threshold; the community response was

Apache Pekko, a fork of Akka 2.6 under Apache 2.0 [34]. Pekko is actively maintained and is a reasonable choice, but a platform intended for a twenty-year infrastructure lifetime should weigh the governance history explicitly.

Microsoft Orleans. The virtual actor model is arguably a better conceptual match for twins than the classic actor model, because grains are addressed by durable identity, activated on demand, and passivated when idle, which mirrors how twin populations actually behave [18], [19]. Orleans is mature, well documented, and integrated with the .NET ecosystem. Its weaknesses relative to the BEAM are the same two as Akka’s — shared-heap collection and non-preemptive scheduling of user code — plus a weaker story at the constrained edge. The instructive point is that the virtual actor model is available on the BEAM through `Erleans` [20], so the choice is not between the two models but about which runtime the chosen model executes on.

Managed twin platforms. Azure Digital Twins with its Digital Twins Definition Language, and AWS IoT TwinMaker with its entity-component model, both provide a graph or entity model, ingestion connectors, and visualization, with the twin execution semantics handled by the provider. For organizations already committed to one cloud and needing a shadow rather than a true twin, these are frequently the correct answer and this paper does not argue otherwise. The reasons to build instead are specific: custom twin behaviour that does not fit the provider’s model, latency requirements that a round trip to a cloud region cannot meet, air-gapped or regulated deployments, edge autonomy during disconnection, and avoidance of a dependency whose pricing and lifecycle are outside the operator’s control over an asset lifetime measured in decades.

Container-per-twin orchestration. Giving each twin a container and letting Kubernetes provide placement, restart, and discovery is conceptually clean and economically impossible at twin-population scale: container overhead is measured in tens of megabytes and scheduling latency in seconds, against kilobytes and microseconds for a BEAM process. The pattern is viable when twins number in the tens and each is computationally heavy, and collapses by three orders of magnitude. Kubernetes remains entirely appropriate as the substrate on which BEAM *nodes* are deployed, and the two choices are complementary rather than competing.

Conventional service architectures. A stateless service tier over a database, with twin state in rows and behaviour in request handlers, is what most teams will build by default. It is a legitimate design and it has one decisive weakness for this domain: it has no natural representation of a long-lived entity with in-memory state and its own failure domain. Every twin operation becomes a read-modify-write cycle with concurrency control, per-twin state machines become status columns with application-enforced transitions, and the isolation property must be reconstructed from optimistic locking or serializable transactions. It works, and it is the architecture that most twin platforms in production actually use. The claim here is that it requires more code and more vigilance to reach the same properties the BEAM provides structurally.

XIII. LIMITATIONS AND CONTRAINDICATIONS

The following conditions make the BEAM the wrong choice, and an architect encountering them should choose otherwise.

If the dominant workload is numerical rather than coordinative — if the system’s core is a finite-element solver, a computational fluid dynamics model, or continuous large-scale inference, and the entity management around it is comparatively simple — then the substrate should be chosen for the numerics and the coordination handled conventionally. The delegation architecture described in Section VII is appropriate when coordination dominates and numerics are episodic; it is contrived when the proportions are reversed.

If hard real-time guarantees are required within the twin layer itself, the BEAM does not provide them. Its scheduling is soft real-time, and while GRiSP narrows the gap for embedded work, deterministic microsecond-scale response belongs in the programmable logic controller or in dedicated firmware. A twin platform should be explicit that it sits above the deterministic control layer and does not replace it.

If the organization has no BEAM expertise and no path to acquiring it, the risk is real and should not be minimized. The talent pool is small relative to Java, Python, or C# ecosystems; hiring takes longer; and the failure mode of a team learning OTP under delivery pressure is a system that uses processes as objects and supervisors as decoration, obtaining the costs of the model without its benefits. Elixir’s learning curve is gentler than Erlang’s and the tooling is good, but the conceptual shift from shared-state concurrency is genuine and takes months rather than weeks.

If the required integrations exist only as vendor SDKs for other languages — a specific historian, a proprietary control system, a licensed simulation engine — then the integration cost may exceed the architectural benefit. Ports and external services mitigate this without puncturing isolation, but they add operational complexity that must be counted honestly.

Finally, if the system is in truth a digital shadow rather than a digital twin — if data flows automatically from the physical side and returns only through a human decision — then most of the requirements that motivate this architecture do not bind, and a conventional or managed platform will reach an adequate result sooner. The argument in this paper is proportional to the strength of the correspondence obligation, and it should not be applied where that obligation is weak.

XIV. EVALUATION METHODOLOGY

The claims in this paper are architectural, and architectural claims are frequently asserted and rarely tested. This section proposes an evaluation design that would allow them to be falsified. It is stated as a methodology rather than as results, because the corresponding measurements at full scale have not been performed and it would be improper to imply otherwise.

A suitable validation context should have a public specification, a physical simulation of record against which digital state can be checked, an established benchmark scenario, and enough entities to exercise the scaling claims. Water distribution networks satisfy all four. EPANET provides the hydraulic

solver [35], WNTR provides a Python framework for resilience analysis and scenario generation over EPANET models [36], and the C-Town network, introduced through the Battle of the Attack Detection Algorithms, supplies a widely used topology together with documented attack scenarios against the supervisory control layer [37]. The combination permits an unusual experimental arrangement: the hydraulic simulator plays the role of physical ground truth, the twin platform maintains its state estimate from simulated telemetry, and the divergence between the two is directly measurable at every time step, including during injected faults and attacks.

Five measurements follow, each mapped to a claim made above.

Correspondence error under nominal operation — the distribution of divergence between twin state and simulator ground truth — establishes the baseline against which everything else is compared, and tests nothing controversial. The second measurement is correspondence recovery after induced failure: kill a proportion of twin processes, kill a node, and partition the cluster, and measure the time until correspondence is re-established and the maximum divergence observed during recovery. This tests R2 and R4 directly and is the central reliability claim of the paper.

Third, latency dispersion under load: drive telemetry at increasing rates and report the full latency distribution for twin queries, not the mean. The claim under test is that the ninety-ninth and ninety-ninth-point-nine percentiles degrade gracefully rather than collapsing, and the comparison of interest is against a shared-heap runtime under identical load, since that is where the theoretical difference lies.

Fourth, scaling behaviour in entity count: replicate the network topology to generate populations from one thousand to one million twins, and report memory per twin, activation latency, and the point at which registry or supervisor contention becomes the binding constraint. This tests R1 and R5, and would also establish empirically where `PartitionSupervisor` sharding or a virtual actor scheme becomes necessary.

Fifth, upgrade under load: perform a hot code upgrade of the twin module during sustained telemetry and measure correspondence loss, dropped commands, and state migration failures against a rolling-restart baseline. This tests R8 and is the measurement most likely to reveal that the theoretical advantage is smaller in practice than in principle.

An evaluation of this shape would be informative regardless of outcome, and would be more useful to the field than a further qualitative argument. Its principal cost is the comparison implementations, since a single-substrate result establishes feasibility but not advantage.

XV. ADOPTION PATH

For an organization persuaded by the argument but reasonably unwilling to commit an entire platform on it, the risk-managed sequence is to introduce the BEAM where its properties are decisive and its integration surface is smallest.

The natural first component is the twin state and lifecycle layer for a single bounded asset class — one building’s mechanical equipment, one pumping district, one production cell — with existing ingestion and existing visualization retained and integrated over conventional protocols. This exercises the process-per-entity model, the supervision structure, and the persistence discipline on a population large enough to be informative and small enough to be reversible. It also produces the artifact most useful for an internal decision: a working system whose failure behaviour can be demonstrated rather than described.

The second step, once that layer is trusted, is to move ingestion into `Broadway` pipelines, which is where the back-pressure benefit is realized and where most of the operational stability improvement in practice comes from. The third is to add `LiveView` for operational views, which tends to displace a surprising amount of separately maintained front-end code. Clustering, virtual actors, and edge deployment through `Nerves` or `AtomVM` should be deferred until the single-node model is well understood, because each adds failure modes whose diagnosis requires the fluency that the earlier steps build.

Throughout, three practices are worth establishing from the beginning: mailbox lengths and supervisor restart intensities as first-class monitored metrics, since both are leading indicators of the failure modes described in Section VI; `Dialyzer` and the `Elixir` type system in continuous integration; and TLS distribution with certificate authentication from the first clustered deployment rather than retrofitted, because the cookie-based default has a way of surviving into production.

XVI. CONCLUSION

The case for building digital twins on `Elixir` and `OTP` is a case about structural correspondence rather than about performance or fashion. Digital twin systems require many isolated long-lived stateful entities, tolerance of continuous partial failure, runtime lifecycle churn, location transparency, live introspection, and evolution without loss of correspondence. The `BEAM` provides all six as runtime primitives because it was built for a problem — carrier telephony — with the same structure. On other substrates these properties are assembled from a runtime, an orchestrator, a state store, a broker, and a control plane, and the assembly must be maintained. That is the whole of the argument, and it is a substantial one.

It is also a bounded one. The `BEAM` is a poor numerical engine and should delegate that work rather than attempt it. It offers no hard real-time guarantee and belongs above the deterministic control layer, not in place of it. Its distribution layer has a mesh ceiling that a large deployment will meet. Its security defaults reflect an era of trusted networks and require active correction, and the disclosure of a maximum-severity pre-authentication remote code execution vulnerability in its own SSH implementation should end any assumption that the ecosystem is exempt from ordinary vulnerability management. Its reliability reputation rests in part on a widely repeated figure whose evidentiary basis is weaker than its circulation suggests, and the real argument — structural failure

containment, supervisory recovery, continuously exercised recovery paths — is stronger without it.

The recommendation that follows is therefore conditional and specific. Where the correspondence obligation is strong, where entities are numerous and long-lived, where partial failure is continuous, and where the same model must run at the edge and in the centre, the `BEAM` is an unusually well-matched substrate for the coordination and state-custody layer of a digital twin platform, and the numerical layer should be built elsewhere and called into it. Where those conditions do not hold — where the system is in truth a shadow, where numerics dominate, or where the organization has no path to the expertise — the honest recommendation is a different one, and this paper’s argument should not be stretched to cover it.

APPENDIX A ISO 23247 TO OTP MAPPING

TABLE I
REFERENCE ARCHITECTURE ENTITIES AND THEIR OTP REALIZATION

ISO 23247 entity	OTP realization
Observable	Physical asset; no software artifact
Manufacturing Element	
Data Collection and Device Control Entity	Supervised protocol adapter subtree; <code>Broadway</code> pipelines with demand-driven back-pressure; separately supervised and audited command path
Digital Twin Entity	<code>gen_statem</code> per element under <code>DynamicSupervisor</code> ; <code>Registry</code> keyed by durable asset identity; event journal with snapshots
User Entity	<code>Phoenix</code> HTTP and <code>WebSocket</code> surface; <code>LiveView</code> operational views; <code>Phoenix.PubSub</code> notification fabric
Proximity network	Adapter-to-device link (MQTT, OPC UA, Modbus, serial)
Access network	Erlang distribution or brokered link between acquisition and twin subtrees

APPENDIX B REQUIREMENTS SUMMARY

REFERENCES

- [1] J. Armstrong, “Making reliable distributed systems in the presence of software errors,” Ph.D. dissertation, Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.
- [2] J. Armstrong, “Erlang,” *Communications of the ACM*, vol. 53, no. 9, pp. 68–75, 2010.
- [3] J. Armstrong, R. Virding, C. Wikström, and M. Williams, *Concurrent Programming in ERLANG*, 2nd ed. Hemel Hempstead, U.K.: Prentice Hall, 1996.
- [4] F. Cesarini and S. Vinoski, *Designing for Scalability with Erlang/OTP*. Sebastopol, CA, USA: O’Reilly Media, 2016.
- [5] S. Jurić, *Elixir in Action*, 3rd ed. Shelter Island, NY, USA: Manning, 2024.
- [6] F. Hébert, *Learn You Some Erlang for Great Good!* San Francisco, CA, USA: No Starch Press, 2013.
- [7] C. Hewitt, P. Bishop, and R. Steiger, “A universal modular ACTOR formalism for artificial intelligence,” in *Proc. 3rd Int. Joint Conf. Artificial Intelligence (IJCAI)*, 1973, pp. 235–245.
- [8] G. Agha, *Actors: A Model of Concurrent Computation in Distributed Systems*. Cambridge, MA, USA: MIT Press, 1986.

TABLE II
REQUIREMENTS AGAINST BEAM/OTP MECHANISMS

	Requirement	Status and mechanism
R1	Entity multiplicity with isolation	Construction; private-heap processes
R2	State longevity, restart semantics	Engineering; journal plus snapshots
R3	Soft real-time under load	Construction; preemption, per-process GC
R4	Partial failure as normal	Construction; supervision trees
R5	Lifecycle churn	Construction; DynamicSupervisor
R6	Location transparency, mobility	Engineering; Horde, Erleans, Partisan
R7	Live introspection	Construction; sys, tracing, telemetry
R8	Evolvability in place	Engineering; hot loading, code_change
R9	Edge and cloud symmetry	Construction; Nerves, GRiSP, AtomVM
R10	Security posture	Not provided; requires active hardening

- [9] M. Grieves and J. Vickers, “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” in *Transdisciplinary Perspectives on Complex Systems*. Cham, Switzerland: Springer, 2017, pp. 85–113.
- [10] W. Kritzing, M. Karner, G. Traar, J. Henjes, and W. Sihn, “Digital twin in manufacturing: A categorical literature review and classification,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [11] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” *IEEE Trans. Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [12] Z. Lyu, Ed., *Handbook of Digital Twins*. Boca Raton, FL, USA: CRC Press, 2024.
- [13] *Automation Systems and Integration — Digital Twin Framework for Manufacturing — Part 2: Reference Architecture*, ISO 23247-2:2021, International Organization for Standardization, Geneva, Switzerland, 2021.
- [14] *Automation Systems and Integration — Digital Twin Framework for Manufacturing — Part 4: Information Exchange*, ISO 23247-4:2021, International Organization for Standardization, Geneva, Switzerland, 2021.
- [15] *Digital Twin — Concepts and Terminology*, ISO/IEC 30173:2023, International Organization for Standardization and International Electrotechnical Commission, Geneva, Switzerland, 2023.
- [16] *Asset Administration Shell for Industrial Applications — Part 1: Asset Administration Shell Structure*, IEC 63278-1, International Electrotechnical Commission, Geneva, Switzerland, 2023.
- [17] G. Shao, S. Frechette, and V. Srinivasan, “An analysis of the new ISO 23247 series of standards on digital twin framework for manufacturing,” in *Proc. ASME Manufacturing Science and Engineering Conf. (MSEC)*, New Brunswick, NJ, USA, 2023.
- [18] S. Bykov, A. Geller, G. Kliot, J. R. Larus, R. Pandya, and J. Thelin, “Orleans: Cloud computing for everyone,” in *Proc. 2nd ACM Symp. Cloud Computing (SOCC)*, 2011.
- [19] P. Bernstein, S. Bykov, A. Geller, G. Kliot, and J. Thelin, “Orleans: Distributed virtual actors for programmability and scalability,” Microsoft Research, Tech. Rep. MSR-TR-2014-41, 2014.
- [20] T. Sloughter et al., “Erleans: Erlang framework for building distributed applications with virtual actors,” Software project. [Online]. Available: <https://github.com/erleans/erleans>
- [21] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Proc. 13th Int. Symp. Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2011, pp. 386–400.
- [22] C. S. Meiklejohn, H. Miller, and P. Alvaro, “PARTISAN: Scaling the distributed actor runtime,” in *Proc. USENIX Annu. Technical Conf. (ATC)*, Renton, WA, USA, 2019, pp. 63–76.
- [23] C. Meiklejohn and P. Van Roy, “Lasp: A language for distributed, coordination-free programming,” in *Proc. 17th Int. Symp. Principles and Practice of Declarative Programming (PPDP)*, 2015, pp. 184–195.
- [24] N. Chechina, P. Trinder, A. Ghaffari, R. Green, K. Lundin, and R. Virding, “The design of scalable distributed Erlang,” in *Proc. Symp. Implementation and Application of Functional Languages (IFL)*, Oxford, U.K., 2012.
- [25] G. Candea and A. Fox, “Crash-only software,” in *Proc. 9th Workshop on Hot Topics in Operating Systems (HotOS)*, 2003.
- [26] J. Gray, “Why do computers stop and what can be done about it?” Tandem Computers, Tech. Rep. 85.7, 1985.
- [27] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [28] S. Gilbert and N. Lynch, “Perspectives on the CAP theorem,” *Computer*, vol. 45, no. 2, pp. 30–36, 2012.
- [29] M. Cronqvist, “The nine nines,” presented at the Erlang Factory, San Francisco Bay Area, CA, USA, 2010.
- [30] National Vulnerability Database, “CVE-2025-32433: Erlang/OTP SSH server pre-authentication remote code execution,” National Institute of Standards and Technology, 2025.
- [31] Unit 42, Palo Alto Networks, “Erlang/OTP SSH vulnerability analysis and exploits observed in the wild,” Threat research report, 2025.
- [32] M. Tóth and I. Bozó, “Static analysis of complex software systems implemented in Erlang,” in *Central European Functional Programming School (CEFP)*, Lecture Notes in Computer Science, vol. 7241. Berlin, Germany: Springer, 2012, pp. 440–498.
- [33] D. Horpácsi, P. Bereczky, and S. Thompson, “Program equivalence in an untyped, call-by-value functional language with uncurried functions,” *Journal of Logical and Algebraic Methods in Programming*, vol. 128, 2022.
- [34] The Apache Software Foundation, “Apache Pekko: A fork of Akka 2.6 predating the Business Source License change,” Project documentation. [Online]. Available: <https://pekko.apache.org>
- [35] L. A. Rossman, “EPANET 2 users manual,” U.S. Environmental Protection Agency, Cincinnati, OH, USA, Rep. EPA/600/R-00/057, 2000.
- [36] K. A. Klise, M. Bynum, D. Moriarty, and R. Murray, “A software framework for assessing the resilience of drinking water systems to disasters with an example earthquake case study,” *Environmental Modelling & Software*, vol. 95, pp. 420–431, 2017.
- [37] R. Taormina et al., “Battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks,” *Journal of Water Resources Planning and Management*, vol. 144, no. 8, 2018.
- [38] Ericsson AB, “Erlang Run-Time System Application (ERTS) reference manual,” Erlang/OTP documentation, 2026.
- [39] Nerves Project, “Nerves: Craft and deploy bulletproof embedded software in Elixir,” Project documentation. [Online]. Available: <https://nerves-project.org>
- [40] AtomVM Project, “AtomVM: A lightweight implementation of the BEAM for microcontrollers,” Project documentation. [Online]. Available: <https://www.atomvm.net>
- [41] Stritzinger GmbH, “GRiSP: Bare-metal Erlang for embedded systems,” Project documentation. [Online]. Available: <https://www.grisp.org>
- [42] S. Cunningham, J. Valim, et al., “Nx: Multi-dimensional arrays and numerical definitions for Elixir,” Software project. [Online]. Available: <https://github.com/elixir-nx/nx>
- [43] Dashbit, “Broadway: Concurrent and multi-stage data ingestion and data processing with Elixir,” Project documentation. [Online]. Available: <https://elixir-broadway.org>
- [44] C. McCord, B. Tate, and S. Jurić, *Programming Phoenix LiveView*. Raleigh, NC, USA: Pragmatic Bookshelf, 2024.
- [45] The Elixir Team, “Elixir v1.20 released,” elixir-lang.org, Jun. 2026. [Online]. Available: <https://elixir-lang.org/blog/2026/06/03/elixir-v1-20-0-released/>